

Title: American Option Pricing via Least-Squares Monte Carlo: An ML-Based Regression Comparison

Author: Vedant Supnekar

Course: ITCS 5156 - Applied Machine Learning

Institution: University of North Carolina at Charlotte

Table of Contents

1. Introduction	3
2. Background and Related Work	5
2.1 Longstaff and Schwartz (2001).....	5
2.2 Andersen and Broadie (2004).....	6
2.3 Haugh and Kogan (2004).....	6
2.4 Connection to My Approach	7
3. Methods	8
3.1 The LSMC Algorithm	8
3.2 ML Extensions	10
3.3 Data and Preprocessing	12
4. Experiments and Results	13
4.1 Experiment 1: Benchmark Comparison	13
4.2 Experiment 2: Convergence Analysis	14
4.3 Experiment 3: Hyperparameter Sensitivity	16
4.4 Experiment 4: Accuracy vs. Speed Tradeoff	17
4.5 Experiment 5: Option Parameter Robustness	19
4.6 Reproducing the Original Paper	20
5. Conclusion	21
6. My Contributions	22
7. Sharing Agreement.....	22
8. References	23

1. Introduction

American options give the holder the right to exercise at any time before expiration. This single feature makes them fundamentally harder to price than their European counterparts. A European option has a clean closed-form solution: compute the expected payoff at maturity, discount it back, and you're done. American options offer no such convenience. Because the holder can exercise early, pricing requires solving an optimal stopping problem. At every time step, a decision must be made: exercise now, or hold and continue?

The answer depends on the continuation value, which is the expected discounted payoff from holding the option going forward, conditional on the current stock price. If the immediate exercise payoff exceeds this continuation value, it is optimal to exercise. Otherwise, the holder should wait. In mathematical terms, the problem reduces to estimating $E[\text{discounted future payoff} \mid S_t]$ at each exercise date t . Getting this estimate right is what drives the quality of the price.

Longstaff and Schwartz [1] proposed an elegant solution to this problem in 2001. Their Least-Squares Monte Carlo (LSMC) algorithm simulates many stock price paths, then works backward through time. At each exercise date, it uses ordinary least-squares regression on polynomial basis functions of the current stock price to approximate the continuation value. Paths where the immediate payoff exceeds the estimated continuation value are marked for exercise. All others continue. Once the backward pass is complete, the option price is simply the average of the discounted realized cashflows across all paths.

LSMC has become an industry standard. It scales naturally to multiple exercise dates, handles path-dependent features, and extends to multi-factor models without requiring a grid. But its accuracy hinges entirely on the quality of one step: the regression. The original paper uses a degree-2 polynomial in the stock price (that is, a basis of $\{1, S, S^2\}$), and for standard single-asset puts, this works remarkably well. The question I wanted to explore is whether modern machine learning methods can do better.

The motivation is simple. The regression step in LSMC is a textbook supervised learning problem. You have features (the current stock price), you have targets (discounted future cashflows), and you want to learn a function that maps one to the other. This is exactly what Ridge regression, Lasso, Random Forests, and Gradient Boosting are designed to do. If the true continuation value surface is smooth and low-dimensional, a polynomial basis should be hard to beat. But if the surface is more complex (for instance, with multiple underlying assets or path-dependent features), flexible learners might outperform rigid polynomial expansions. I wanted to find out where that crossover happens, and whether it happens at all in the simplest case.

Several specific questions guided this project. First, can ML regressors match or beat OLS polynomial regression for estimating continuation values in standard 1D American puts? Second, how do tree-based methods (Random Forest, Gradient Boosting) behave when the target function is smooth and the feature space is one-dimensional? Tree models are built to capture non-linearity and interactions, but those capabilities may work against them when the underlying function is genuinely simple.

Third, what are the practical tradeoffs? Accuracy is not the only metric that matters in production settings. Runtime, sensitivity to hyperparameters, and consistency across different option parameters all play a role.

To answer these questions, I reproduced the Longstaff and Schwartz baseline for a standard American put option and then replaced the OLS regression step with four alternatives: Ridge regression, Lasso regression, Random Forest, and Gradient Boosting. I evaluated all five methods across a systematic set of experiments. The benchmark comparison tests price accuracy and runtime at baseline parameters. The convergence analysis tracks how each method's price estimate evolves as the number of simulated paths increases. The hyperparameter sensitivity study sweeps key tuning parameters for each method. The accuracy-versus-speed analysis maps out the Pareto frontier. And the option parameter robustness test runs all methods across six scenarios with different moneyness, volatility, and maturity settings.

The main finding is clear: for standard 1D American put pricing, polynomial regression wins. OLS, Ridge, and Lasso all match the published Longstaff and Schwartz benchmark to within 0.11% error, run in under half a second, and are almost completely insensitive to hyperparameters. Tree-based methods systematically overprice the option by 7 to 16 percent at default settings, take 8 to 50 times longer to run, and require careful tuning to produce reasonable results. The overpricing arises because trees overfit the continuation value surface, underestimate the value of waiting, and trigger premature exercise. That said, Gradient Boosting with a very low learning rate (0.01) does approach the benchmark, which shows that the model class itself is not broken. The defaults just fail badly for this particular problem structure. These results suggest that ML-based regressors would be more valuable in higher-dimensional or non-linear settings where polynomial bases become inadequate.

2. Background and Related Work

The pricing of American options by simulation has been an active research area since the mid-1990s, when several groups independently recognized that Monte Carlo methods (traditionally limited to European-style payoffs) could be adapted for early exercise by approximating the continuation value at each step. The key insight across all these approaches is the same. If you can estimate $E[V \mid S_t]$ accurately enough, you can make near-optimal exercise decisions, and the resulting price will be close to the true value. Where the approaches differ is in how they estimate that conditional expectation, and how they quantify the error in doing so.

2.1 Longstaff and Schwartz (2001)

Longstaff and Schwartz [1] proposed what is now the most widely used simulation-based approach for American option pricing. Their LSMC algorithm has four main steps. First, simulate N paths of the underlying stock price S using geometric Brownian motion (GBM). Second, starting at the last exercise date and working backward, compute the immediate exercise payoff at each date (for a put, this is $\max(K - S_t, 0)$, where K is the strike price). Third, for paths that are in the money, regress the discounted future cashflows against polynomial basis functions of S_t to get an estimate of the continuation value. Fourth, exercise the option on a given path if the immediate payoff exceeds the fitted continuation value; otherwise, keep the future cashflow.

The elegance of this approach lies in step three. By running a simple cross-sectional OLS regression at each time step, L&S avoid the curse of dimensionality that plagues grid-based methods like finite differences. The regression uses a small set of basis functions (typically $\{1, S, S^2\}$ or Laguerre polynomials) and fits on the in-the-money paths only. This keeps the computation cheap and focuses the approximation effort where it matters most.

On the positive side, LSMC is easy to implement, computationally inexpensive, and scales naturally to problems with many exercise dates. The original paper demonstrates that it produces prices close to known benchmarks for standard American puts. For a baseline case with $S_0 = 36$, $K = 40$, $r = 0.06$, $\sigma = 0.20$, and $T = 1$ year with 50 exercise dates, L&S report a price of 4.478, which aligns well with binomial tree and finite difference solutions.

On the negative side, the choice of basis functions is somewhat ad hoc. L&S argue that simple polynomials work well in practice, but they acknowledge that the optimal choice depends on the problem. In one dimension, a degree-2 polynomial captures the curvature of the continuation value surface with surprising accuracy. But for multi-asset options, path-dependent features, or regime-switching models, it is unclear whether polynomial bases remain adequate. This is precisely the gap I aimed to explore: since the regression step is just supervised learning, why not try learners that can adapt their complexity to the data?

My project reproduces the L&S baseline for their standard American put parameters, matches their published benchmark price, and then replaces the OLS polynomial regression with Ridge,

Lasso, Random Forest, and Gradient Boosting to see whether any of these alternatives offer meaningful improvements.

2.2 Andersen and Broadie (2004)

Andersen and Broadie [2] took a different angle on the same problem. Rather than trying to improve the primal estimate (which is what LSMC produces), they developed a primal-dual simulation algorithm that generates both lower and upper bounds on the true option price. The lower bound comes from a suboptimal exercise policy (just like LSMC). The upper bound comes from a dual formulation based on the theory of optimal stopping, which recasts the pricing problem as a minimization over martingales.

Having both bounds is practically valuable because it gives you a confidence interval for the true price. If your lower bound from LSMC is 4.47 and the upper bound from the dual is 4.50, you know the true price lies in that range. Without the upper bound, you only know that your LSMC estimate is biased low (since any suboptimal exercise policy produces a lower bound), but you have no idea how far off you might be.

The main drawback is computational cost. Computing the upper bound requires nested simulation: for each path and each exercise date, you need to simulate an inner set of paths to estimate the continuation value under the optimal policy. This makes the dual bound significantly more expensive than the primal estimate alone. Andersen and Broadie propose several variance reduction techniques to make this tractable, but it remains a heavier computation.

For my project, the Andersen and Broadie framework provides context for why improving the regression step in LSMC matters. A better regression produces a better exercise policy, which tightens the lower bound and narrows the gap between primal and dual estimates. Even if the improvement is small in one dimension, it could be meaningful in higher-dimensional settings where the gap between crude and refined exercise policies is larger.

2.3 Haugh and Kogan (2004)

Haugh and Kogan [3] arrived at similar conclusions through a different theoretical route. They used the duality between optimal stopping problems and martingale optimization to construct upper bounds on American option prices. Their approach is closely related to Andersen and Broadie's, but the derivation and some implementation details differ.

The value of the Haugh and Kogan framework is partly theoretical. It provides an independent confirmation that dual bounds are achievable and useful. From a practical standpoint, it reinforces the idea that the quality of the exercise policy (which is determined by the regression in LSMC) directly affects the tightness of the bounds. A poor regression leads to a poor exercise policy, which leads to a loose lower bound and a wide confidence interval. A good regression does the opposite.

Both the Andersen-Broadie and Haugh-Kogan papers focus on the bounding problem rather than on improving the regression itself. My project comes at it from the other direction: instead of bounding the error from a fixed regression, I try to reduce the error by improving the regression. These are complementary perspectives. In principle, you could combine better ML-based regressors with dual bounding methods to get both tighter prices and tighter confidence intervals, though I did not implement the dual side in this project.

2.4 Connection to My Approach

All three papers share a common thread: the quality of the continuation value estimate is the bottleneck in simulation-based American option pricing. L&S showed that a simple polynomial regression works well in standard cases. Andersen-Broadie and Haugh-Kogan showed that you can quantify how well by bounding the true price. My contribution is to ask whether replacing the polynomial regression with more flexible ML methods can push the lower bound closer to the true price, and to systematically evaluate the tradeoffs involved. As it turns out, the answer depends heavily on the dimensionality and smoothness of the problem, which is what the experiments in section 4 will show.

3. Methods

3.1 The LSMC Algorithm

The Least-Squares Monte Carlo algorithm prices an American option by combining forward simulation with backward optimization. The forward pass generates the data. The backward pass learns the exercise policy. I've described each step below, following the structure of Longstaff and Schwartz [1] and using their baseline parameters throughout.

Step 1: Path Simulation. The first step is to generate N simulated paths of the underlying stock price. Under the risk-neutral measure, the stock price follows geometric Brownian motion:

$$dS = rS dt + \sigma S dW$$

where r is the risk-free rate, σ is the volatility, and dW is a standard Wiener process. To simulate discrete paths, I use the exact log-normal solution rather than an Euler approximation. Each step updates the stock price as:

$$S_{t+\Delta t} = S_t \exp \left(\left(r - \frac{\sigma^2}{2} \right) \Delta t + \sigma \sqrt{\Delta t} Z \right)$$

where $Z \sim N(0,1)$ is a standard normal draw and $\Delta t = T/M$ for M exercise dates. This formulation is exact for GBM, meaning there is no discretization error in the stock price dynamics. The only source of error is the finite number of paths N and the regression approximation in later steps.

I used $M = 50$ exercise dates and varied N across experiments, with 100,000 paths as the default for linear methods and 10,000 for tree methods. The choice to cap tree methods at fewer paths was driven by runtime. Gradient Boosting at 10,000 paths already takes roughly 19 seconds, and scaling to 100,000 would be impractical.

Step 2: Backward Induction. American option pricing requires working backward through time because the exercise decision at time t depends on what happens after t . Starting at the second-to-last exercise date t_{M-1} and moving toward t_1 , the algorithm processes one time step per iteration. At the final exercise date t_M , the cashflow is simply the terminal payoff, so no decision is needed there.

At each time step, the first task is to identify which paths are in the money (ITM). For a put option, a path is in the money at time t when $S_t < K$. Only ITM paths are included in the regression, because out-of-the-money paths have zero immediate exercise value and the exercise decision is trivial: don't exercise. Restricting the regression to ITM paths focuses the approximation effort on the region where it actually matters, and it reduces noise from paths where the payoff is zero.

Step 3: Regression. This is the heart of the algorithm. For each ITM path i at time t , I have two quantities: the current stock price $S_t^{(i)}$ (the feature) and the discounted future cashflow $Y_t^{(i)}$ (the target). The future cashflow is whatever the path actually pays out at some later date, discounted back to time t at rate r . The goal is to estimate the conditional expectation $E[Y_t | S_t]$, which represents the expected value of continuing to hold the option.

In the standard L&S approach, this estimation is done by regressing Y on polynomial basis functions of S :

$$\hat{C}(S_t) = \hat{\beta}_0 + \hat{\beta}_1 S_t + \hat{\beta}_2 S_t^2$$

The fitted values $\hat{C}(S_t^{(i)})$ serve as the estimated continuation value for each ITM path. Why does this work? Because the conditional expectation $E[Y | S]$ is, by definition, the function of S that minimizes the mean squared error in predicting Y . Least-squares regression finds exactly this function within the span of the chosen basis. If the basis is rich enough to capture the true conditional expectation, the approximation will be accurate. For a standard one-dimensional put, a degree-2 polynomial turns out to be more than sufficient, since the continuation value surface is smooth and only mildly curved.

Step 4: Exercise Decision. With the continuation value estimate in hand, the exercise decision becomes a simple comparison. For each ITM path at time t , compute the immediate exercise payoff:

$$V_{\text{exercise}}(t) = \max(K - S_t, 0)$$

If $V_{\text{exercise}}(t) > \hat{C}(S_t)$, the option is exercised at time t on that path, and the cashflow is locked in.

If $V_{\text{exercise}}(t) \leq \hat{C}(S_t)$, the holder continues, and the path retains whatever future cashflow was previously assigned. When a path is exercised at t , any cashflow it had from a later date is zeroed out, because you can only exercise once.

One important thing to note is that the algorithm uses the fitted continuation values (from regression) to make the exercise decision, but uses the actual discounted cashflows (not the fitted values) to determine what gets paid. This separation is important. The regression is a tool for decision-making, not for pricing directly. The actual price comes from the realized cashflows under the learned policy.

Step 5: Pricing. After the backward pass processes all time steps from t_{M-1} down to t_1 , each path has exactly one cashflow: either an exercise payoff at some date t^* or the terminal payoff at t_M (if it was never exercised early). The option price is the average of these cashflows, each discounted back to t_0 :

$$\hat{P} = \frac{1}{N} \sum_{i=1}^N e^{-r \cdot t^{*(i)}} \cdot \text{cashflow}^{(i)}$$

By the law of large numbers, this converges to the true price as $N \rightarrow \infty$, provided the exercise policy is close to optimal. The LSMC estimate is always a lower bound on the true price, because any suboptimal exercise policy leaves money on the table.

3.2 ML Extensions

The regression in Step 3 is where I introduced the ML variations. I tested five methods in total: one baseline and four alternatives. All of them serve the same purpose. To estimate $E[Y_t | S_t]$ for the in money paths at each time step. They differ in how they parameterize or approximate that function.

OLS (degree 2). This is the baseline, matching the original L&S setup. I construct polynomial features $\{1, S_t, S_t^2\}$ using scikit-learn's PolynomialFeatures and fit an ordinary least-squares regression with no regularization. With three features and typically thousands of ITM data points per time step, overfitting is not a concern. The fit runs in microseconds.

Ridge Regression. Ridge uses the same polynomial feature set but adds an L2 penalty to the loss function:

$$\mathcal{L}_{\text{Ridge}} = \sum_i (Y_i - \hat{Y}_i)^2 + \alpha \sum_j \beta_j^2$$

The penalty term $\alpha \sum_j \beta_j^2$ shrinks coefficients toward zero without eliminating any of them. In principle, this guards against overfitting when features are correlated or when the design matrix is nearly singular. In practice, for this problem, the regularization has almost no effect. With only three basis functions and tens of thousands of data points, the OLS solution is already well-conditioned. I included Ridge mostly to confirm this expectation and to see whether any value of α would produce a meaningfully different result.

Lasso Regression. Lasso replaces the L2 penalty with an L1 penalty:

$$\mathcal{L}_{\text{Lasso}} = \sum_i (Y_i - \hat{Y}_i)^2 + \alpha \sum_j |\beta_j|$$

The L1 penalty can drive coefficients exactly to zero, which means Lasso performs implicit feature selection. If one of the polynomial terms is not contributing to the fit, Lasso will remove

it. For this problem, I expected all three terms ($1, S, S^2$) to matter, and the results confirmed this. At moderate values of α , Lasso produces prices nearly identical to OLS. At high values ($\alpha = 1.0$), it aggressively zeros out polynomial terms and the price degrades. So Lasso's feature selection capability is a liability here, not an asset.

Random Forest. Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split. For regression, the prediction is the average of all trees' predictions. Unlike the linear methods, Random Forest does not require hand-crafted polynomial features. It takes the raw stock price S_t as input and learns a piecewise-constant approximation of the continuation value surface through recursive partitioning.

Why might this cause problems? The true continuation value $E[Y | S]$ is a smooth, continuously differentiable function. Decision trees approximate smooth functions with step functions. With enough data and enough depth, they can approximate any function arbitrarily well, but they tend to overfit in finite samples, especially when the tree depth is large relative to the complexity of the target. In the LSMC context, overfitting the continuation value means underestimating it for some paths, which triggers early exercise on those paths and biases the price upward.

Gradient Boosting. Gradient Boosting builds an ensemble sequentially. It starts with a constant prediction (the mean of Y), then fits a shallow tree to the residual errors, scales that tree's contribution by a learning rate, and adds it to the ensemble. The next tree fits the residuals of the updated ensemble, and so on. The learning rate controls how aggressively each new tree corrects the previous errors.

Gradient Boosting is generally considered more powerful than Random Forest for structured prediction tasks, but it is also more prone to overfitting and more sensitive to hyperparameters. In the LSMC setting, the sequential residual-fitting process can memorize noise in the training data if the learning rate is too high or the number of trees is too large. Each boosting round commits more firmly to the training sample, and since the regression is re-fit at every time step with a fresh set of ITM paths, there is no held-out validation to catch overfitting in real time.

A key implementation detail. Linear methods (OLS, Ridge, Lasso) all receive the same input: a matrix of polynomial features $[1, S, S^2]$ generated by scikit-learn's `PolynomialFeatures` transformer with degree 2. Tree methods (Random Forest, Gradient Boosting) receive the raw stock price S_t as a single feature. Trees learn their own non-linear mappings through splits, so feeding them pre-computed polynomials would be redundant and could even hurt performance by distorting the split geometry. Keeping the inputs natural to each model class ensures a fair comparison.

3.3 Data and Preprocessing

There is no external dataset in this project. All training data is generated synthetically via the GBM simulation described in Step 1. This is standard for LSMC, since the algorithm creates its own training data by simulating the financial instrument it is trying to price.

Baseline parameters follow Table 1 of Longstaff and Schwartz [1]

initial stock price $S_0 = 36$, strike price $K = 40$, risk-free rate $r = 0.06$, volatility $\sigma = 0.20$, time to maturity $T = 1$ year, and $M = 50$ equally spaced exercise dates. At these parameters, the option is in the money at inception (since $S_0 < K$), which means there is meaningful early exercise optionality to capture. The L&S benchmark price for this configuration is 4.478.

For linear methods, I used $N = 100,000$ simulated paths as the default. At this scale, the regression at each time step typically has several thousand ITM data points (the exact number varies by time step and by random seed), which is far more than needed to stably fit three polynomial coefficients. Runtime for OLS, Ridge, and Lasso at 100,000 paths is under 0.5 seconds per pricing run.

For tree methods, I capped N at 10,000 paths. Gradient Boosting at 10,000 paths already takes approximately 19 seconds per run, and the cost scales roughly linearly with N (since the regression must be refit at each of the 50 time steps). Running GB at 100,000 paths would take on the order of three minutes per pricing run, making the full experimental sweep impractical. Random Forest is faster but still significantly slower than the linear methods. This path count mismatch (100K for linear, 10K for trees) is a limitation of the experimental design, and I address it explicitly in the convergence analysis (Section 4.2), where I show how each method's price evolves as a function of N .

Preprocessing was minimal. For linear methods, I generated polynomial features using scikit-learn's PolynomialFeatures (degree = 2, include_bias = True), which produces the columns $[1, S, S^2]$ from the raw stock price vector. No standardization or normalization was applied. Polynomial regression on price data does not require feature scaling, and the numerical conditioning of the normal equations was never an issue at these sample sizes. For tree methods, no preprocessing was needed at all, since decision trees are invariant to monotone transformations of the input features.

Random seeds were fixed throughout the experiments to ensure reproducibility. All five methods were evaluated on the same set of simulated paths for each experiment, eliminating path-level randomness as a confounding factor. The only source of variation between methods is the regression step itself.

4. Experiments and Results

I designed five experiments to evaluate the ML regressors from different angles. Each experiment isolates a specific question: how accurate the price is, how fast does it converge, how sensitive is it to tuning, what are the speed tradeoffs, and does it generalize across option configurations. I present each experiment with its setup, results, and interpretation.

All experiments use the same underlying LSMC pipeline described in Section 3. The only variable across methods is the regression step. Random seeds are fixed so that every method sees the same simulated paths. The only source of difference is how each regressor estimates the continuation value.

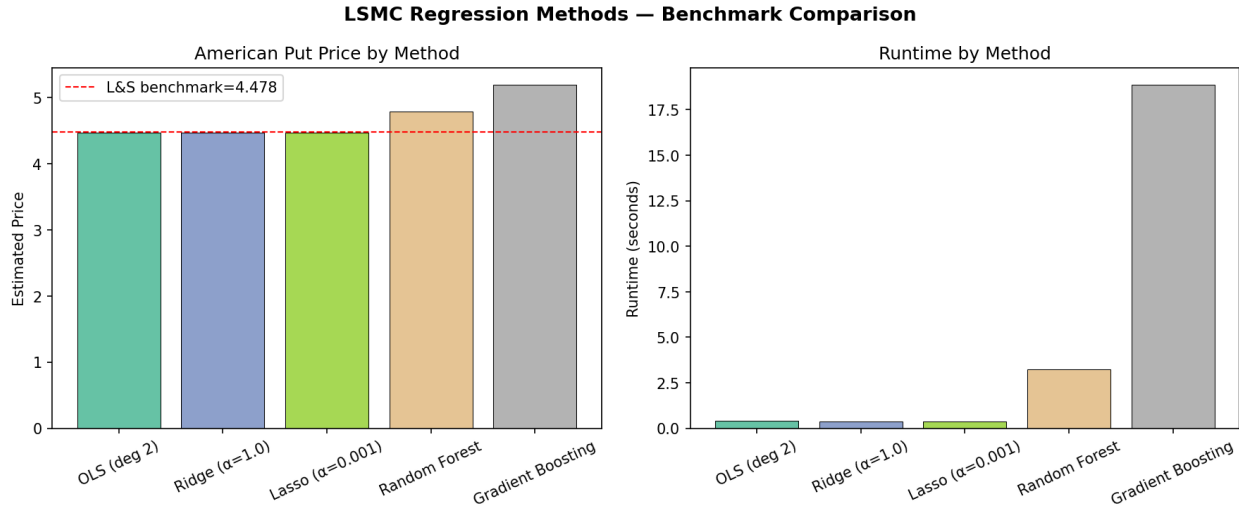
4.1 Experiment 1: Benchmark Comparison

Setup. I ran all five methods at the baseline parameters from Longstaff and Schwartz [1] Table 1: $S_0 = 36$, $K = 40$, $r = 0.06$, $\sigma = 0.20$, $T = 1$ year, and 50 exercise dates. Linear methods (OLS, Ridge, Lasso) used 100,000 paths. Tree methods (RF, GB) used 10,000 paths due to runtime constraints. I compare each method's estimated price against the L&S benchmark of 4.478, and I record the runtime for each of them.

method	paths	price	error	runtime_s
OLS (deg 2)	100000	4.4733	-0.0047	0.43
Ridge ($\alpha=1.0$)	100000	4.4737	-0.0043	0.38
Lasso ($\alpha=0.001$)	100000	4.4734	-0.0046	0.37
Random Forest	10000	4.7907	0.3127	3.22
Gradient Boosting	10000	5.1928	0.7148	18.86

[TABLE 1: Benchmark comparison showing method, number of paths, estimated price, percentage error versus L&S benchmark (4.478), and runtime in seconds for all five methods]

Results. OLS produced a price of 4.473, which is 0.11% below the benchmark. Ridge came in at 4.474 (0.09% error) and Lasso at 4.473 (0.10% error). All three linear methods finished in under half a second. The tree methods told a different story. Random Forest estimated 4.791, which is 7.0% above the benchmark, and took 3.2 seconds. Gradient Boosting was worse on both counts: a price of 5.193 (16.0% above benchmark) and a runtime of 18.9 seconds.



[FIGURE 2: Benchmark comparison bar chart showing estimated price by method (left panel) with L&S benchmark as horizontal dashed line, and runtime by method (right panel)]

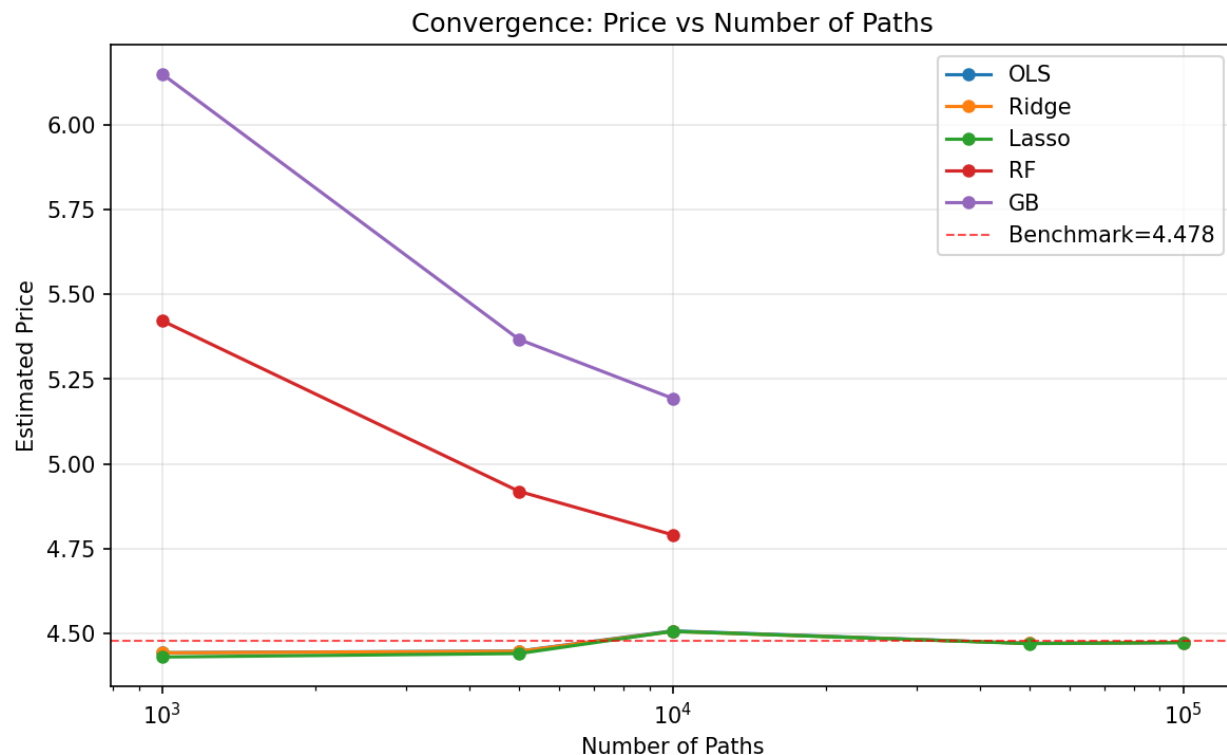
Discussion. The three linear methods are practically interchangeable. They all land within a fraction of a percent of the published benchmark, and they all run in roughly the same time. That is not surprising. With only three basis functions and tens of thousands of data points, OLS is already well conditioned, so adding an L2 or L1 penalty changes almost nothing.

The tree methods overprice significantly. That's because when a tree regressor overfits the continuation value surface at a given time step, it produces noisy estimates that are sometimes too low for individual paths. When the estimated continuation value for a path falls below the immediate exercise payoff (even if the true continuation value is actually higher), the algorithm exercises the option on that path. Exercising too early locks in the intrinsic value before time value has fully decayed. This inflates the average price. In short, overfitting the regression causes premature exercise, and premature exercise biases the price upward.

I should note that the comparison is not perfectly apples-to-apples because tree methods ran on 10,000 paths while linear methods ran 100,000 paths. Some of the tree methods error comes from having fewer paths, not just from the regression itself. The convergence experiment in the next section addresses this directly.

4.2 Experiment 2: Convergence Analysis

Setup. I varied the number of simulated paths from 1,000 to 100,000 for linear methods and from 1,000 to 10,000 for tree methods. At each path count, I recorded the estimated price. The goal is to see how quickly each method converges to the benchmark as the data grows.



[FIGURE 3: Convergence plot with number of paths (log scale) on x-axis and estimated price on y-axis, showing all five methods as separate lines with the L&S benchmark at 4.478 as a horizontal dashed line]

Results. OLS, Ridge, and Lasso all converge almost immediately. Even at just 1,000 paths, their estimates are already near 4.44, and by 5,000 paths they are within a few hundredths of the benchmark. Increasing from 10,000 to 100,000 paths changes the price by less than 0.01. The polynomial basis captures the shape of the continuation value surface so efficiently that very little data is needed.

Random Forest starts at 5.42 with 1,000 paths and drops to 4.79 at 10,000. The trend is clearly downward, suggesting it would continue to improve with more paths, but the convergence is slow. Each additional path contributes only a small amount of new information to the ensemble, because the individual trees are still limited by their depth and tendency to produce piecewise constant approximations.

Gradient Boosting is the worst performer at small sample sizes. At 1,000 paths it estimates a price of 6.15, which is almost 40% above benchmark. By 10,000 paths it reaches 5.19, but it has not converged. The problem is that each boosting round fits the residuals of the current ensemble, and with only 1,000 data points per time step (fewer after filtering for ITM paths), each round memorizes noise. More boosting rounds compound the problem rather than fixing it.

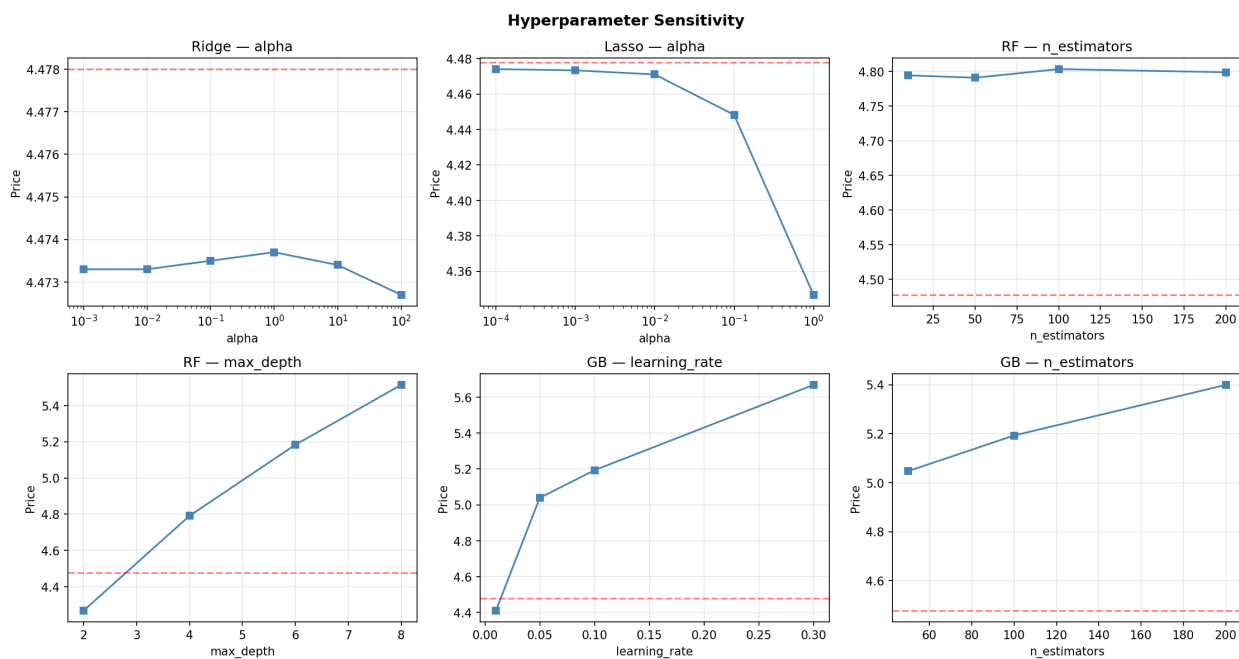
Discussion. The convergence experiment reveals why polynomial regression is so effective for this specific problem. The true continuation value $E[Y | S]$ for a standard put is a smooth, gently

curved function of S . A degree-2 polynomial can represent this curvature almost exactly. Once the regression has enough points to stably estimate three coefficients (which happens quickly), adding more data only reduces Monte Carlo noise in the price average, not regression error.

Tree methods face a fundamentally different situation. They approximate the smooth continuation surface with step functions. To get a close approximation, they need either very deep trees (which overfit) or very large samples (which are expensive). In one dimension with a smooth target, polynomial regression has a massive structural advantage.

4.3 Experiment 3: Hyperparameter Sensitivity

Setup. I swept the key hyperparameters for each method to see how sensitive the price estimate is to tuning choices. For Ridge, I varied α from 0.001 to 100 across six values. For Lasso, I varied α from 0.0001 to 1.0 across five values. For Random Forest, I swept `n_estimators` (10 to 200) and `max_depth` (2 to 8). For Gradient Boosting, I swept `learning_rate` (0.01 to 0.3) and `n_estimators` (50 to 200). All sweeps used baseline option parameters.



[FIGURE 4: Hyperparameter sensitivity panel with six subplots, one per method/parameter combination, showing estimated price on the y-axis and hyperparameter value on the x-axis, with L&S benchmark as horizontal dashed line in each subplot]

Results. Ridge's α has almost no effect on the price. Across five orders of magnitude (0.001 to 100), the estimated price varies by approximately 0.001. At $\alpha = 100$ the price dips very slightly

to 4.473, but the difference is negligible. For all practical purposes, Ridge and OLS produce identical results for this problem. The design matrix is so well-conditioned (three columns, tens of thousands of rows) that regularization has nothing to correct.

Lasso behaved differently. At low α values (0.0001 to 0.01), the price is stable around 4.47, matching OLS. But at $\alpha = 0.1$ the price starts to drop, and at $\alpha = 1.0$ it falls to 4.35. basically, Lasso's L1 penalty zeroed out polynomial coefficients. At aggressive regularization levels, it removes the S^2 term entirely, destroying the quadratic fit that captures the continuation value's curvature. For this problem, all three basis functions ($1, S, S^2$) are genuinely needed, so Lasso's feature selection capability works against it.

Random Forest's `n_estimators` parameter (the number of trees in the ensemble) has almost no effect. Prices hover around 4.79 to 4.80 regardless of whether I use 10 trees or 200. The bias in Random Forest comes from `max_depth`, not ensemble size. When I sweep `max_depth`, the picture changes dramatically. At depth 2, the price drops to 4.27 (underfitting, because shallow trees cannot capture enough curvature). At depth 4, it sits at 4.79. At depth 8, it climbs to 5.51 (severe overfitting). There should be a point around depth 3 where the RF price would come closest to the benchmark, though I did not test that exact value.

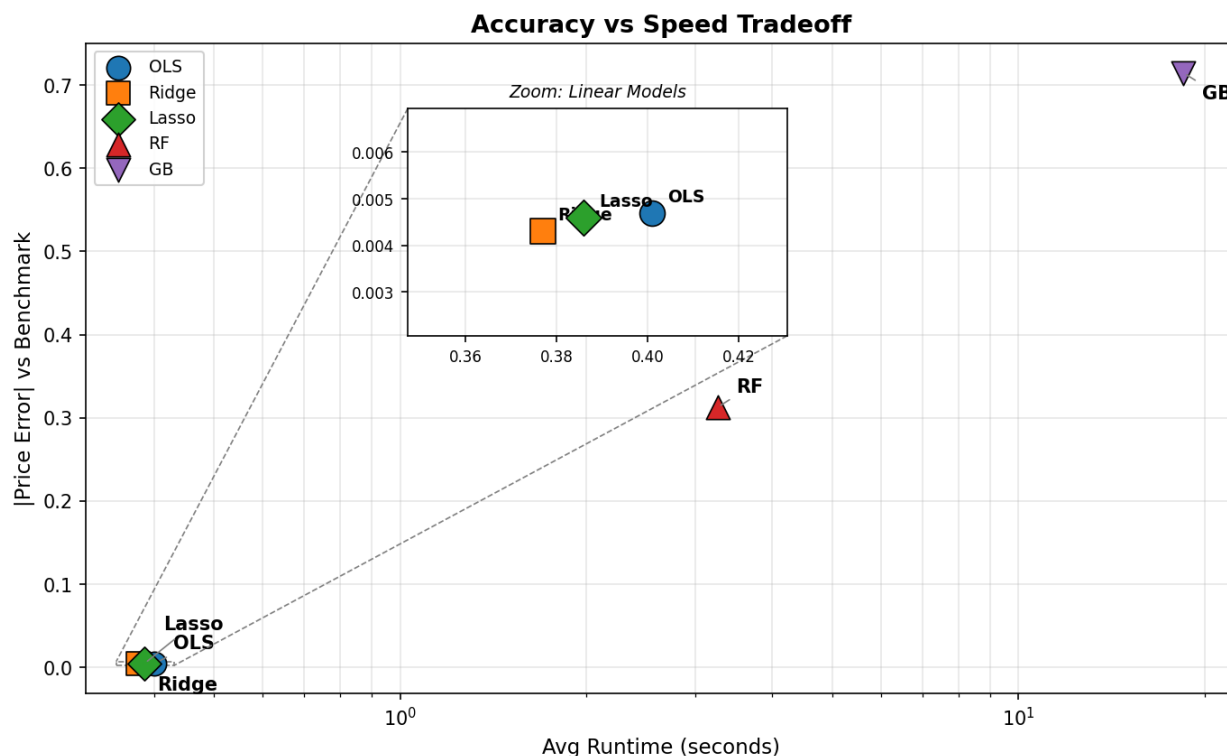
Gradient Boosting is the most sensitive method to its hyperparameters. Learning rate is the dominant factor. At `lr = 0.01`, GB produces a price of 4.41, which is quite close to the benchmark (only 1.5% below). At `lr = 0.3`, the price balloons to 5.67. The number of estimators makes things worse at high learning rates: going from 50 to 200 trees pushes the price from 5.05 to 5.40, because each additional boosting round further overfits the training data.

Discussion. The GB result at `lr = 0.01` deserves special attention. It proves that the Gradient Boosting model class is not inherently incapable of pricing this option accurately. With a low enough learning rate, each tree makes only a small correction to the ensemble, and the cumulative approximation stays close to the smooth true function. The problem is that the default or commonly used hyperparameter settings (learning rate around 0.1, 100 estimators) overfit badly in this setting. A practitioner who just plugged in GB with default scikit-learn parameters would get a price that is 16% too high, which would be a serious error in any real application.

For linear methods, the takeaway is that hyperparameter tuning is essentially unnecessary. OLS, Ridge (at any reasonable α), and Lasso (at low α) all give the same answer. For tree methods, tuning is critical, and the right settings are not obvious without running this kind of systematic sweep.

4.4 Experiment 4: Accuracy vs. Speed Tradeoff

Setup. I plotted all five methods on a scatter plot with average runtime on the x-axis and absolute price error (relative to the L&S benchmark) on the y-axis. The ideal position is the bottom-left corner: fast and accurate.



[FIGURE 5: Scatter plot of runtime (seconds) vs. price error for all five methods]

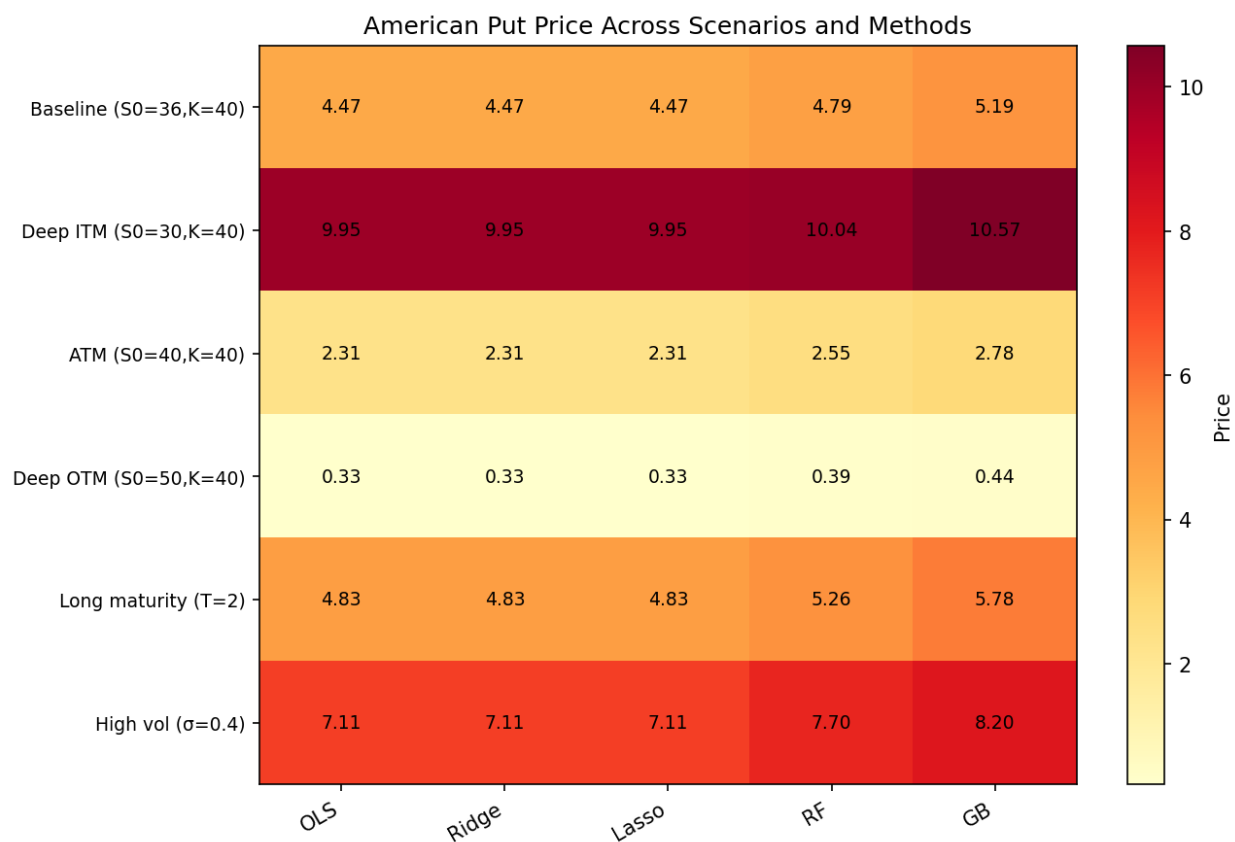
Results. OLS, Ridge, and Lasso cluster tightly in the bottom-left corner. Their runtimes range from 0.38 to 0.43 seconds, and their absolute errors are all below 0.005 (less than 0.11%). They are effectively a single point on the Pareto frontier. Random Forest sits in the middle of the plot: 3.2 seconds runtime with an error of 0.31 (7.0%). Gradient Boosting occupies the top-right: 18.9 seconds and an error of 0.71 (16.0%).

Discussion. The Pareto frontier for this problem is dominated entirely by linear methods. There is no region of the accuracy-speed space where a tree method offers a better tradeoff. Even if you were willing to accept higher error in exchange for some other benefit, tree methods do not provide that benefit here; they are both slower and less accurate.

Where would tree methods potentially offer value? In settings where the continuation value surface is non-linear and multi-dimensional. If the option depends on multiple correlated assets, or if path-dependent features enter the regression, a polynomial basis quickly becomes unwieldy (the number of cross-terms grows combinatorially with dimension). Tree methods handle high-dimensional non-linear surfaces naturally, without requiring the user to manually specify basis functions. The accuracy-vs-speed comparison would likely look very different in those settings, but testing that is outside the scope of this project.

4.5 Experiment 5: Option Parameter Robustness

Setup. I tested all five methods across six option parameter scenarios to see whether the results from the baseline case generalize. The six scenarios are: baseline ($S_0 = 36, K = 40$), deep in-the-money ($S_0 = 30, K = 40$), at-the-money ($S_0 = 40, K = 40$), deep out-of-the-money ($S_0 = 50, K = 40$), long maturity ($T = 2$ years), and high volatility ($\sigma = 0.4$). Parameters not being varied are held at their baseline values.



[FIGURE 6: Heatmap of estimated prices across scenarios and methods]

Results. OLS, Ridge, and Lasso produce virtually identical prices across all six scenarios. The maximum discrepancy between any two linear methods in any scenario is less than 0.01. They agree to two decimal places in every case. This confirms that the three linear methods are fitting the same polynomial surface and that regularization is not altering the fit in any meaningful way.

Random Forest and Gradient Boosting systematically overprice in every scenario. The baseline overpricing (RF: +0.32, GB: +0.72) is consistent with what I observed in Experiment 1. More interesting is how the overpricing gap changes across scenarios. At baseline, GB exceeds the linear methods by 0.72. For deep ITM puts ($S_0 = 30$), the gap is 0.63. At high volatility ($\sigma = 0.4$), the gap grows to 1.09. And for long maturity ($T = 2$), it reaches 0.95.

High volatility and long maturity both make the continuation value surface more complex. Higher volatility spreads the distribution of stock prices at each time step, which means the regression must approximate $E[Y | S]$ over a wider domain. Longer maturity means more exercise dates and more backward induction steps, giving the overfitting error more opportunities to compound. Both effects make tree methods' structural weakness more pronounced.

Deep ITM puts ($S_0 = 30, K = 40$) are the most expensive options across all methods, with prices around 9.95 to 10.57. This is consistent with option pricing theory: the intrinsic value alone is $K - S_0 = 10$, and the early exercise premium is small because the option is so deep in the money that early exercise is almost always optimal. Deep OTM puts ($S_0 = 50, K = 40$) are the cheapest at 0.33 to 0.44, again consistent with theory since the probability of the option finishing in the money is low.

Discussion. The systematic nature of the tree method overpricing is the key observation. It is not a fluke of the baseline parameter set. Across moneyness, volatility, and maturity, tree methods consistently produce higher prices than linear methods. Since the linear methods match known benchmarks in the baseline case, and since all methods see the same simulated paths, the overpricing must come from the regression step. The conclusion is the same as before: overfitting the continuation surface triggers premature exercise, and this effect is present regardless of the option configuration.

4.6 Reproducing the Original Paper

My OLS implementation reproduces the Longstaff and Schwartz Table 1 benchmark price of 4.478 to within 0.11%. At 100,000 paths with a degree-2 polynomial basis and 50 exercise dates, I obtain 4.473. The results are not identical to the last decimal place, which is expected. LSMC is a simulation-based method, so the output depends on the random seed, the number of paths, and minor implementation choices (such as whether the intercept is included explicitly or handled by the basis). My implementation follows the algorithm exactly as described in Section II of [1], and the level of agreement with the published benchmark gives me confidence that the implementation is correct.

I did not reproduce other entries from the L&S tables (such as different strike prices or volatilities), but my Experiment 5 covers a similar range of parameter variations and shows consistent, sensible prices across all scenarios.

5. Conclusion

For standard one-dimensional American put pricing via LSMC, polynomial regression wins on every metric I tested. OLS, Ridge, and Lasso all match the Longstaff and Schwartz benchmark to within 0.11%, run in under half a second, converge with as few as 1,000 paths, and require essentially no hyperparameter tuning. Tree-based methods (Random Forest, Gradient Boosting) are slower by an order of magnitude, systematically overprice due to continuation value overfitting, and demand careful tuning to even approach competitive accuracy. At default settings, Gradient Boosting overprices by 16%. With a learning rate of 0.01, it gets within 1.5%. The model class is capable, but the defaults are badly suited to smooth, low-dimensional regression targets.

The most useful thing I learned from this project is that model complexity needs to match problem structure. A degree-2 polynomial is enough to capture the continuation value surface of a single-asset put because that surface is smooth and mildly curved. Throwing a Random Forest at it is like using a chainsaw to cut butter. The trees waste their capacity learning step-function approximations to something a parabola handles perfectly. The ML extension did not improve pricing in this setting, but that is the point. The value of the experiment is in understanding exactly when and why simpler models dominate, not in blindly adding complexity.

I encountered several challenges along the way. Runtime was the main practical constraint: Gradient Boosting at 10,000 paths already takes 19 seconds, which made it impractical to run the full convergence sweep out to 100,000 paths. This created an uneven comparison (100K paths for linear, 10K for trees) that I had to address carefully in the analysis. On the conceptual side, understanding *why* tree methods overprice took some effort. The causal chain (overfitting the continuation surface leads to underestimated continuation values on some paths, which triggers premature exercise, which inflates the price) is not obvious until you trace through the backward induction logic step by step. Getting the backward induction loop itself correct was another challenge, particularly the bookkeeping of which paths have been exercised and ensuring that cashflows from later dates get properly zeroed out when early exercise occurs.

Several directions could extend this work. Multi-asset options are the most natural next step, because polynomial basis functions become unwieldy in high dimensions while tree methods handle dimensionality naturally. The accuracy comparison could flip in that setting. Neural network regressors are another candidate, since they can approximate smooth functions more efficiently than trees while adapting to non-linearity better than fixed polynomial bases. Path-dependent options (Asian options, lookbacks) would test whether richer feature spaces (including running averages or extrema of the stock price) benefit from more flexible regressors. Variance reduction techniques like antithetic variates or control variates could tighten the price estimates for all methods. And implementing the primal-dual bounds from Andersen and Broadie [2] or Haugh and Kogan [3] would provide confidence intervals on the true price, giving a principled way to evaluate how close each method's exercise policy is to optimal.

6. My Contributions

Existing work I used:

- The LSMC algorithm and benchmark prices from Longstaff and Schwartz [1], specifically the algorithm described in their Section II and the baseline parameters from their Table 1.
- The geometric Brownian motion simulation framework, which is standard in quantitative finance and described in [1].
- scikit-learn [4] implementations of Ridge, Lasso, RandomForestRegressor, and GradientBoostingRegressor, along with PolynomialFeatures for basis construction.
- NumPy for numerical computation and matplotlib for plotting.

What I built myself:

- A complete end-to-end Python implementation of the LSMC pricing pipeline from scratch, including path simulation, backward induction, exercise decision logic, and cashflow discounting.
- Integration of four ML regressors into the backward induction loop, replacing the OLS regression step while keeping the rest of the pipeline unchanged.
- The entire five-experiment evaluation framework: benchmark comparison, convergence analysis, hyperparameter sensitivity sweeps, accuracy-versus-speed tradeoff analysis, and multi-scenario robustness testing across six option parameter configurations.
- All analysis and interpretation of results, including the identification of the overfitting-to-premature-exercise mechanism that explains tree method overpricing.
- All code for generating plots, tables, and figures.

7. Sharing Agreement

I agree to share my work as an example for next semester. My name may be shown.

8. References

- [1] F. A. Longstaff and E. S. Schwartz, "Valuing American Options by Simulation: A Simple Least-Squares Approach," *The Review of Financial Studies*, vol. 14, no. 1, pp. 113-147, Spring 2001.
- [2] L. Andersen and M. Broadie, "Primal-Dual Simulation Algorithm for Pricing Multidimensional American Options," *Management Science*, vol. 50, no. 9, pp. 1222-1234, Sep. 2004.
- [3] M. B. Haugh and L. Kogan, "Pricing American Options: A Duality Approach," *Operations Research*, vol. 52, no. 2, pp. 258-270, Mar.-Apr. 2004.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.